STUDENT PERFORMANCE TREND ANALYSIS

Python Activity-Based Learning (ABL) Assignment — Sample Implementation

1. Problem Statement

Develop a Python program to compare student assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students based on their overall average.

2. Objectives

• Store marks for multiple students, subjects, and examinations.

• Calculate total marks, overall average, and change between the first and final examinations.

• Identify subjects scoring below 50 in every examination.

• Classify students using clearly defined performance criteria.

• Display a readable report in the terminal.

3. Assumptions and Classification Rules

Rule	Criteria
Excellent	Average >= 85
Good	Average >= 70 and < 85
Average	Average >= 50 and < 70
Needs Improvement	Average >= 40 and < 50
At Risk	Average < 40
Consistently weak subject	Below 50 marks in every exam

4. Algorithm

1. Start and load the student marks for each subject across three examinations.

2. For each student, calculate totals for Exam 1, Exam 2, and the Final Exam.

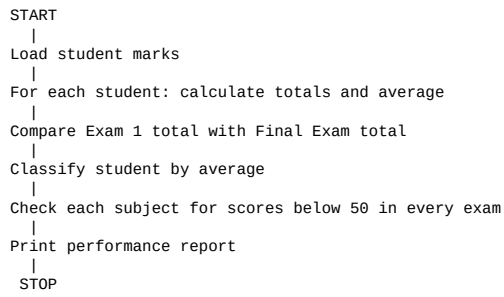
3. Calculate overall average and the change from Exam 1 total to Final Exam total.

4. Classify the student using the overall average.

5. For each subject, check whether every exam score is below 50.

6. Print the summary table and subject-wise trends.

5. Flowchart



6. Pseudocode

```
for each student:
    exam1_total = sum(first exam marks)
    exam2_total = sum(second exam marks)
    final_total = sum(final exam marks)
    average = sum(all subject marks) / number of marks
    change = final_total - exam1_total
    assign category based on average
    for each subject:
```

```
        if every exam mark < 50: record as weak
print summary, subject trends, and weak subjects
```

7. Python Source Code

Save the separate source file as **student_performance_analysis.py** and run it with Python 3. The code below is the implementation used for the sample output.

```
"""
Student Performance Trend Analysis
Compare exam scores, measure improvement/decline,
identify consistently weak subjects, and classify students.
"""

PASS_MARK = 40
WEAK_SUBJECT_MARK = 50

students = {
    "Ananya": {
        "Maths": [65, 72, 80],
        "Physics": [58, 55, 62],
        "Chemistry": [70, 74, 78],
    },
    "Rahul": {
        "Maths": [82, 76, 71],
        "Physics": [68, 60, 55],
        "Chemistry": [74, 72, 70],
    },
    "Sravani": {
        "Maths": [35, 42, 48],
        "Physics": [45, 38, 44],
        "Chemistry": [50, 48, 46],
    },
    "Kiran": {
        "Maths": [90, 92, 95],
        "Physics": [85, 88, 91],
        "Chemistry": [87, 90, 94],
    },
}


def classify_student(average):
    """Classify a student based on overall average marks."""
    if average >= 85:
        return "Excellent"
    elif average >= 70:
        return "Good"
    elif average >= 50:
        return "Average"
    elif average >= PASS_MARK:
        return "Needs Improvement"
    return "At Risk"


def analyze_student(subject_scores):
    """Return subject trends, weak subjects, average, and trend."""
    subject_averages = {}
    weak_subjects = []
    all_scores = []

    for subject, scores in subject_scores.items():
        subject_averages[subject] = sum(scores) / len(scores)
        all_scores.extend(scores)

        # Weak if score is below WEAK_SUBJECT_MARK in every examination.
        if all(mark < WEAK_SUBJECT_MARK for mark in scores):
            weak_subjects.append(subject)

    first_total = sum(scores[0] for scores in subject_scores.values())
    last_total = sum(scores[-1] for scores in subject_scores.values())
    improvement = last_total - first_total
    overall_average = sum(all_scores) / len(all_scores)

    if improvement > 0:
        trend = "Improved"
    elif improvement < 0:
        trend = "Declined"
    else:
        trend = "No Change"

    return {
        "average": overall_average,
        "first_total": first_total,
        "last_total": last_total,
        "improvement": improvement,
        "trend": trend,
        "weak_subjects": weak_subjects,
        "subject_averages": subject_averages,
    }


def print_report():
    print("=" * 94)
    print("                       STUDENT PERFORMANCE TREND ANALYSIS")
    print("=" * 94)
    print("Exams compared: Midterm 1 -> Midterm 2 -> Final | Weak subject: below 50 in every exam")
    print("-" * 94)
    print(f"{'Student':<12}{'Exam 1':>10}{'Exam 2':>10}{'Final':>10}{'Change':>10}{'Average':>11}{'Category':>20}")
    print("-" * 94)

    analyses = {}
    for name, subjects in students.items():
        result = analyze_student(subjects)
```

```
        analyses[name] = result
        print(
            f"{name:<12}{result['first_total']:>10}{sum(v[1] for v in subjects.values()):>10}"
            f"{result['last_total']:>10}{result['improvement']:>+10}"
            f"{result['average']:>11.2f}{classify_student(result['average']):>20}"
        )

    print("-" * 94)
    print("\nSUBJECT-WISE TREND (Exam 1 -> Exam 2 -> Final)")
    for name, subjects in students.items():
        print(f"\n{name}:")
        for subject, scores in subjects.items():
            change = scores[-1] - scores[0]
            direction = "Improved" if change > 0 else "Declined" if change < 0 else "No Change"
            print(f"  {subject:<12}: {scores[0]:>3} -> {scores[1]:>3} -> {scores[2]:>3} | Change {change:+3} | {direction}")

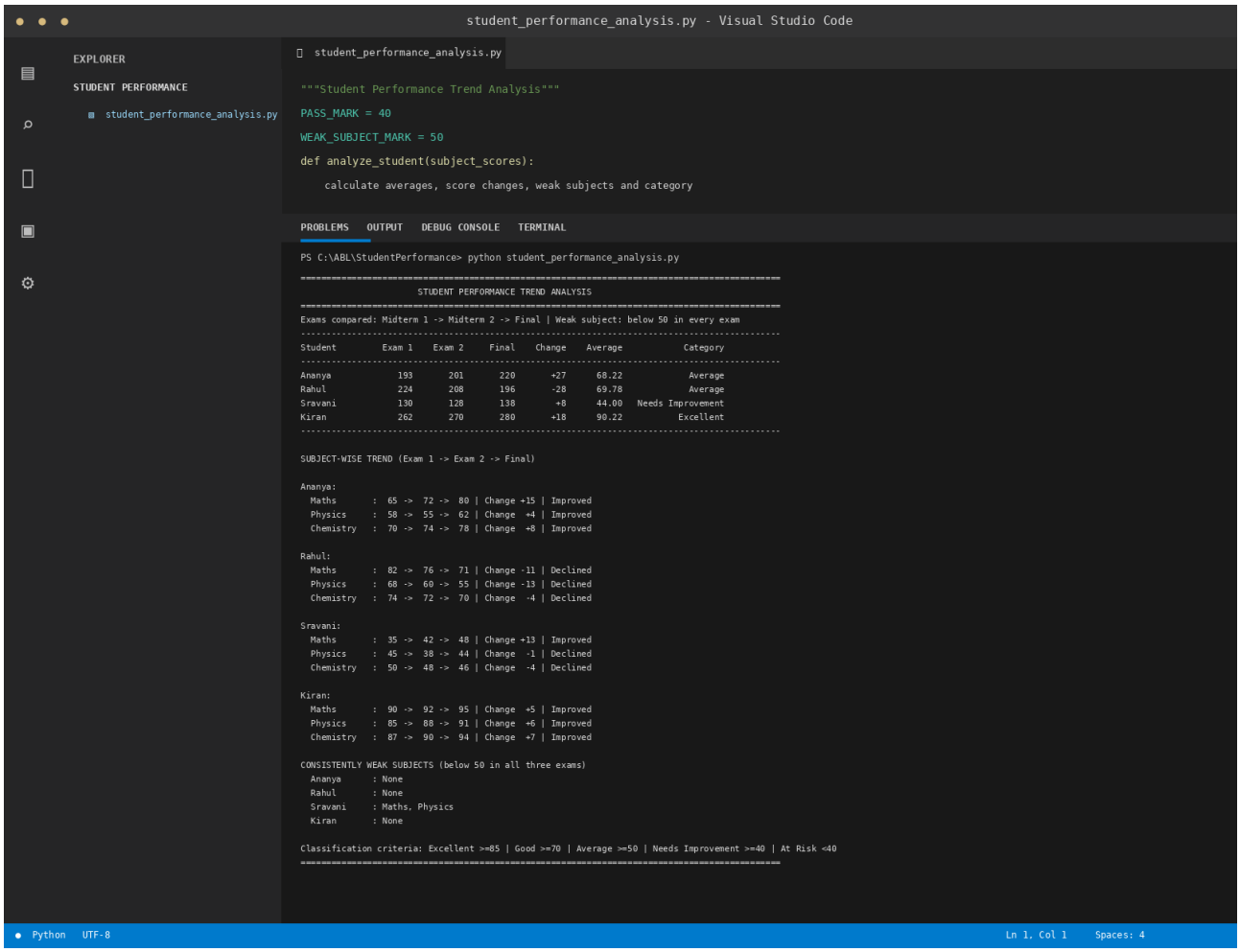
    print("\nCONSISTENTLY WEAK SUBJECTS (below 50 in all three exams)")
    for name, result in analyses.items():
        weak = ", ".join(result["weak_subjects"]) if result["weak_subjects"] else "None"
        print(f"  {name:<12}: {weak}")

    print("\nClassification criteria: Excellent >=85 | Good >=70 | Average >=50 | Needs Improvement >=40 | At Risk <40")
    print("=" * 94)


if __name__ == "__main__":
    print_report()
```

8. Test Data and Executed Output

The sample uses four students, three subjects (Maths, Physics, Chemistry), and three assessments (Midterm 1, Midterm 2, Final). The image is a VS Code-style screenshot mockup displaying the output from running the supplied Python file.



Terminal output (text copy)

```
================================================================================================
                        STUDENT PERFORMANCE TREND ANALYSIS
================================================================================================
Exams compared: Midterm 1 -> Midterm 2 -> Final | Weak subject: below 50 in every exam
------------------------------------------------------------------------------------------------
Student         Exam 1    Exam 2     Final    Change    Average            Category
------------------------------------------------------------------------------------------------
Ananya             193       201       220       +27      68.22             Average
Rahul              224       208       196       -28      69.78             Average
Sravani            130       128       138        +8      44.00   Needs Improvement
Kiran              262       270       280       +18      90.22           Excellent
------------------------------------------------------------------------------------------------

SUBJECT-WISE TREND (Exam 1 -> Exam 2 -> Final)

Ananya:
  Maths       :  65 ->  72 ->  80 | Change +15 | Improved
  Physics     :  58 ->  55 ->  62 | Change  +4 | Improved
  Chemistry   :  70 ->  74 ->  78 | Change  +8 | Improved

Rahul:
  Maths       :  82 ->  76 ->  71 | Change -11 | Declined
  Physics     :  68 ->  60 ->  55 | Change -13 | Declined
  Chemistry   :  74 ->  72 ->  70 | Change  -4 | Declined

Sravani:
  Maths       :  35 ->  42 ->  48 | Change +13 | Improved
  Physics     :  45 ->  38 ->  44 | Change  -1 | Declined
  Chemistry   :  50 ->  48 ->  46 | Change  -4 | Declined

Kiran:
  Maths       :  90 ->  92 ->  95 | Change  +5 | Improved
  Physics     :  85 ->  88 ->  91 | Change  +6 | Improved
  Chemistry   :  87 ->  90 ->  94 | Change  +7 | Improved
CONSISTENTLY WEAK SUBJECTS (below 50 in all three exams)
```

```
  Ananya      : None
  Rahul       : None
  Sravani     : Maths, Physics
  Kiran       : None

Classification criteria: Excellent >=85 | Good >=70 | Average >=50 | Needs Improvement >=40 | At Risk <40
================================================================================================
```

9. Test Cases

Test	Scenario	Expected behavior
1	Scores rise across exams	Trend shows Improved
2	Final total lower than first total	Trend shows Declined
3	All subject scores below 50	Subject listed as consistently weak
4	Average >= 85	Excellent classification
5	Average below 40	At Risk classification

10. Conclusion

The program summarizes assessment trends, quantifies score changes, flags subjects that remain below the chosen threshold, and assigns performance categories. The marks and thresholds are illustrative; replace them with your faculty's required rules if different.